

BAX-POC Azure 인프라 구축 작업 기록

Terraform 코드 검토 · Azure 배포 · Linux VM 환경 구성 및 점검

프로젝트: BeingAX AI Agent POC (bax-poc)

환경: BaxPOC-DEV (Azure 구독: d6ec08db-4ce1-4ed6-b16c-b4ff61504b4d)

작성일: 2026-06-08

1. 개요

본 문서는 BAX-POC 프로젝트의 Azure 인프라를 Terraform으로 검토·배포하고, 배포된 Linux VM에 접속하여 운영에 필요한 유틸리티(Azure CLI, kubectl)를 설치 및 점검한 전체 과정을 정리한 작업 기록입니다.

- 작업 범위: Terraform 코드 구조 분석 → Azure 배포 실행 → VM 접속 → 운영 도구 설치/점검
- 대상 환경: environments/BaxPOC-DEV (azurerm provider 4.63.0, Terraform >= 1.6)
- 대상 구독 ID: d6ec08db-4ce1-4ed6-b16c-b4ff61504b4d

2. Terraform 코드 구조 검토

2.1 모듈 구성

modules/ 하위에 리소스 그룹, 네트워크(VNet/Subnet/NSG/Route Table/NAT Gateway/DNS), 컴퓨팅(Linux VM, Windows VM, AKS, Container Registry), AI(Foundry, OpenAI, AI Search), 데이터(PostgreSQL), 프라이빗 엔드포인트 등 총 18개 재사용 모듈이 정의되어 있으며, environments/BaxPOC-DEV/main.tf에서 이를 조합해 실제 배포 구성을 정의합니다.

2.2 BaxPOC-DEV 환경에서 호출된 주요 모듈

구분	리소스명	비고
리소스 그룹	bax-poc-dev-app-rg / network-rg / ai-rg	용도별 3개 분리
네트워크	bax-poc-dev-vnet (10.130.0.0/24)	mgmt/pe/db/aks 4개 서브넷
Linux VM	bax-poc-d-vm (Standard_D4s_v5)	관리/점프박스 용도, Public IP 활성화
AKS	bax-poc-dev-aks	system(sysnp01) + user(usernp01) 노드풀, private_cluster_enabled=false
Container Registry	baxpocdcr (Standard)	admin 활성화, 퍼블릭 액세스 허용
AI Foundry	bax-poc-dev-msf (kind=AIServices)	Private Endpoint + 3개 Private DNS Zone 연결
PostgreSQL	bax-poc-dev-psql (Flexible Server, PG 18)	Private Endpoint 구성

2.3 검토 중 식별된 주요 사항

- provider "azurerm"에 subscription_id가 명시되어 있지 않아 Azure CLI 로그인 컨텍스트의 구독으로 배포됨
- foundry 모듈의 location이 "EastUS2"로 하드코딩되어 있어 다른 리소스(var.location=koreacentral)와 리전이 상이함 — 의도 여부 확인 필요
- VM(admin_password) 및 PostgreSQL(administrator_password) 비밀번호가 main.tf에 평문("Auditax12#\$")으로 하드코딩되어 있음 — 보안상 Key Vault 또는 sensitive 변수로 분리 권장
- backend 설정이 없어 상태 파일(terraform.tfstate)이 로컬에 저장됨 — 협업 시 원격 백엔드 구성 검토 필요

- lifecycle, random/timestamp 등 매 실행마다 diff를 유발할 수 있는 요소는 발견되지 않음 — 기본적으로 변경분만 반영되는 구조

3. Azure 배포 실행

3.1 사전 준비 사항

- Azure 구독 ID 확보 및 해당 구독에 대한 Contributor 이상 권한
- Azure CLI 설치 및 로그인 (provider가 별도 인증정보 없이 features{}만 정의 → CLI 컨텍스트에 의존)
- Terraform 1.6 이상 설치
- custom_subdomain_name(예: bax-poc-dev-msf) 등 전역 고유 이름의 중복 여부 사전 확인
- 리전별 리소스 할당량(Quota) 및 Microsoft.CognitiveServices, Microsoft.ContainerService 등 Resource Provider 등록 여부 확인

3.2 실행 명령어 (environments/BaxPOC-DEV 디렉터리 기준)

Terraform은 현재 디렉터리의 .tf 파일을 루트 모듈로 인식하고, main.tf에서 모듈을 상대경로(../modules/...)로 참조하므로 반드시 environments/BaxPOC-DEV 폴더에서 실행해야 합니다.

```
# 1) Azure 로그인 및 구독 전환
az login
az account set --subscription "d6ec08db-4ce1-4ed6-b16c-b4ff61504b4d"
az account show # 대상 구독 ID로 전환되었는지 확인

# 2) 작업 디렉터리 이동
cd "...\\bax-poc\\environments\\BaxPOC-DEV"

# 3) Terraform 초기화 / 검증
terraform init
terraform validate

# 4) 실행 계획 확인 후 적용
terraform plan -out=tfplan
terraform apply tfplan
```

3.3 트러블슈팅 — terraform init "empty directory"

최초 실행 시 저장소 루트(bax-poc)에서 terraform init을 수행하여 "Terraform initialized in an empty directory!" 메시지가 출력됨. 이는 에러가 아니라 해당 위치에 .tf 파일이 없다는 안내로, environments/BaxPOC-DEV로 이동 후 재실행하여 정상 초기화함.

3.4 배포 결과

- environments/BaxPOC-DEV/main.tf에 정의된 전체 리소스(리소스 그룹 3종, VNet/Subnet, Linux VM, AKS, ACR, AI Foundry+PE, PostgreSQL+PE, Private DNS Zone 등) 배포 완료
- 배포 완료 후 terraform.tfstate에 각 리소스의 상태 정보(민감 정보 포함)가 저장됨을 확인 — 별도 보안 관리 필요

4. Linux VM 접속 및 환경 점검

4.1 접속 정보

항목	값
VM 이름	bax-poc-d-vm
크기	Standard_D4s_v5
계정	azureuser
인증 방식	비밀번호 인증 (disable_password_authentication = false)
네트워크	subnet_mgmt (10.130.0.0/27), Public IP 활성화

※ 비밀번호는 environments/BaxPOC-DEV/main.tf에 평문으로 정의되어 있으며, 운영 전환 시 Key Vault 연동 등으로 분리 관리할 것을 권장함.

4.2 초기 환경 점검 결과

VM 접속 직후 홈 디렉터리(~) 점검 결과, 기본 셸 설정 파일(.bashrc, .profile 등) 외 별도 구성 파일은 없는 초기 상태였으며, az CLI 및 kubectl 모두 미설치 상태로 확인됨.

```
azureuser@bax-poc-d-vm:~$ az login
az: command not found
azureuser@bax-poc-d-vm:~$ kubectl
Command 'kubectl' not found ...
```

4.3 Azure CLI 설치

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

4.4 kubectl 설치

apt 패키지 저장소를 통한 설치 시 "E: Unable to locate package kubectl" 오류가 발생하여, 공식 바이너리를 직접 다운로드하는 방식으로 설치를 완료함.

```
# 공식 바이너리 직접 다운로드 (채택한 방법)
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)"
"/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/
kubectl version --client
```

(대안) 공식 Kubernetes apt 저장소 등록 또는 Azure CLI 내장 명령(az aks install-cli) 사용 가능.

4.5 AKS 접속 절차 (정리)

```
az login
az account set --subscription "d6ec08db-4ce1-4ed6-b16c-b4ff61504b4d"
az aks get-credentials --resource-group <app-rg 이름> ♻
```

```
--name bax-poc-dev-aks --overwrite-existing  
kubectl get nodes -o wide  
kubectl cluster-info
```

개별 노드 디버깅이 필요한 경우(직접 SSH는 기본 비활성화):

```
kubectl debug node/〈노드이름〉 -it ₩  
--image=mcr.microsoft.com/dotnet/runtime-deps:8.0 -- chroot /host bash
```

5. 후속 조치 권고 사항

- [검증] VM에서 각 서비스 Private Endpoint DNS 정상 해석 여부 확인 (nslookup), AKS 노드 상태(kubectl get nodes), PostgreSQL 접속 테스트
- [AI 모델] AI Foundry 계정은 생성 완료 상태이며, 실제 모델 배포(azurerm_cognitive_deployment 또는 포털)는 별도 작업으로 진행 필요
- [AKS 연동] ACR(baxpocdcr)과 AKS 연동 (az aks update --attach-acr) 후 애플리케이션 이미지 빌드/배포
- [DB 초기화] PostgreSQL에 POC용 데이터베이스/스키마 생성
- [보안 정리] 코드에 평문으로 포함된 admin_password / administrator_password를 Key Vault 또는 sensitive 변수로 분리, ACR/AKS의 퍼블릭 액세스 설정 재검토
- [모니터링] kubernetes 모듈의 log_analytics_workspace_id 변수에 워크스페이스를 연결하여 AKS 모니터링 구성